

(find this project’s code on my GitHub at <https://github.com/Dylan-Brown/CVTailor>!)

CVTailor

Agentic workflows meet deterministic RAG. CVTailor is an automated, high-throughput pipeline that ingests job descriptions and outputs a tailored resume and cover letter. Built with verifiable fact-checking, it ensures high-quality personalization without LLM hallucinations.

The Core Philosophy: Deterministic RAG

CVTailor eliminates the uncertainty of standard Retrieval-Augmented Generation (RAG). Instead of relying on similarity searches and hoping the LLM stays within bounds, this pipeline enforces strict, deterministic grounding:

1. **Closed-Corpus Retrieval:** Your resume is extracted once into an atomic, ID-addressed `claims_ledger.json`. No similarity thresholds; the model sees the entire verified truth.
2. **Required Citations:** Every proposed edit must explicitly cite its grounding `claim_id` at generation time.
3. **Deterministic Verification:** Edits without citations are discarded by code—zero LLM calls involved. Edits with citations are strictly verified against the original claim text. Any hallucination or fabrication is a hard, permanent discard.

“*Grounded in truth*” is not an assumption, but rather an auditable property of the pipeline.

Pipeline Architecture (10 Stages)

The pipeline is broken down into modular, standalone scripts (`src/stages/py_stageNN_<name>.py`).

- **00. Normalize Input:** Validates untrusted job description JSONs.
- **01. Score Input (Optional):** Decides which of your resume variants best fits the role.
- **02. Ingest CV Claims:** Parses your resume into verifiable facts (`claims_ledger.json`).
- **03. Research Company:** LLM + targeted web search for company context.
- **04. Analyze Gaps:** Compares the job requirements against your claims.
- **05. Propose Updates:** The LLM generates candidate edits, strictly constrained to your ledger.
- **06. Fact Check (The Gate):** Discards fabrications. Edits must be grounded in reality.
- **07. Evaluate Utility:** Discards edits that are true, but irrelevant to *this specific* job.

- **08. Agentic Review:** The **only manual step**. You (or a pluggable AI reviewer) approve/reject the surviving edits.
- **09. Assemble Materials:** Generates the final tailored `.docx` and `.pdf` files.

Quickstart & Batch Workflow

CVTailor is built to process many applications concurrently.

1. Place your base materials in their respective `config/` folders:
 - `resume_pdfs/<your_resume>.pdf`
 - `resume_template/<template>.docx`
 - `resume_variants/<variant>.json`
 - `cover_letter_sample/<sample>.txt`
 - `cover_letter_template/<template>.docx` (Optional)
2. Fill out `config/applicant_info.json`.
3. Copy `pipeline.example.yaml` to `pipeline.yaml` (gitignored – personal values only) and adjust as needed.
4. Set your credentials (if applicable) in `config/credentials/`. No API keys are needed if you use local models via LM Studio, but `ANTHROPIC_API_KEY` is needed for Claude, and `GEMINI_API_KEY` is needed if you opt to use Gemini.
5. Set up the Browser Extension (Job Data Harvester) found in `extension/chrome/` by visiting `chrome://extensions/`, toggle on “Developer mode” then click “Load unpacked” and select the path to the project’s extension at `/extension/chrome/`; optionally configure the native host bridge in `extension/native_host/` (this will bring the job description `.json` files into `/applications` automatically; otherwise, move manually).
6. (Optional) Configure the always-on File Watcher by setting up `config/file_watcher.json` and registering it via `util/Register-FileWatcher.ps1` (Admin PowerShell) or running `python src/auto_queue/py_file_watcher.py`.
7. Run `python src/checkpoints/py_pipeline_precheck.py` to verify your setup.

Daily Workflow:

1. **Intake:** Drop raw JD JSONs into `applications/` or use the Chrome extension.
2. **Prepare:** Ensure LM Studio is up and running a local server with your preferred model loaded (e.g., `qwen2.5-coder-14b-instruct`). (Note: The File Watcher handles LM Studio warm-up automatically if configured). Run `python run_pipeline.py` to execute stages 0-7 across all pending applications.
3. **Review:** Run `python src/ai_wrappers/py_stage08_reviewer_interface.py` to approve edits — it reads `pipeline.yaml`’s `review_stage_assignee` and dispatches to either the interactive human CLI or the registered Agent backend (Gemini by default). To always review by hand regardless of that setting, run `src/stages/py_stage08_agentic_update_review.py` directly instead.
4. **Assemble:** Run `python src/controls/py_pipeline_assemble.py`

to generate the final PDFs. 5. **Apply:** Submit the application using the generated materials, then file it so `applications/` doesn't just accumulate everything you've ever sent — *(optional but helpful)* `python src/controls/py_post_pipeline_store_applied.py <app_id_prefix>` moves it into `applications/archive/applied/` (or `--cutoff / --revisit / --test` to file it under one of those instead).

(Tip: `python run_pipeline.py --full` chains steps 3-4 plus opening application URLs, intended for once `pipeline.yaml` has an Agent assigned so a batch can run start to finish unattended.)

Advanced Features

- **Local-First AI:** Defaults to local models via **LM Studio** for zero-cost, private processing. Cloud providers (Claude, Gemini) are optional (exception: Gemini will review Stage 8 unless otherwise specified, for convenience).
- **Always-On Automation:** Run the optional File Watcher (`src/auto_queue/py_file_watcher.py`) to automatically process new JDs the moment they are saved, complete with LM Studio warm-ups and cloud drive syncing.
- **Smart Document Assembly:** Replaces text inside your `.docx` templates recursively without breaking your existing formatting, tables, or styles.
- **Auto-Formatting:** Automatically applies progressive typography compression to ensure your resume strictly hits your page limits (e.g., exactly 1 or 2 pages).